Week 9 Final Capstone — American Put Option Pricing

CRR Binomial vs. Neural-Network Pricer vs. RL Exercise Policy

1. Executive Summary

This project brings together a CRR binomial pricer, a neural-network price approximator, and a DQN-based early-exercise policy, and compares all three against each other rather than treating them as three separate assignments. The NN pricer was checked against the binomial benchmark across a shared grid of 168 contracts spanning deep in/out-of-the-money puts, short and long maturities, and low and high volatility, coming in at a mean absolute error of **0.083** (RMSE 0.103) against binomial prices ranging from under $1 to roughly $60. The RL policy was checked two ways: against the binomial-optimal exercise boundary for the one contract it was trained on (62.6% boundary agreement, value 6.21 vs. a binomial optimum of 7.95), and, going further than a single-contract check, run against the full 168-contract grid it was never trained on — where it beat a naive always-hold baseline on only 20% of contracts. Both point the same direction and are discussed directly in Section 9 rather than smoothed over.

2. Problem Statement

An American put gives the holder the right to sell the underlying at strike `K` at any time up to expiry `T`. Pricing it means solving an optimal stopping problem: unlike a European put, there's no closed-form solution, because the optimal exercise decision depends on the entire future path of the stock. This project compares three ways of approaching that — an exact numerical method (binomial tree), a learned function approximator for the price (neural network), and a learned decision policy for the exercise timing itself (reinforcement learning).

3. Financial Background

A put's payoff at any exercise time is `max(K - S, 0)`. American options are worth at least as much as their European counterparts, since the holder has strictly more choices (exercise whenever vs. only at expiry); the difference is the early-exercise premium. For a put, early exercise becomes attractive once the option is deep in-the-money and there isn't much time value left to lose, which is exactly the shape the exercise boundary traces out in Section 8.

4. The Binomial Method

`src/pricing/binomial.py::crr_price` builds a Cox-Ross-Rubinstein recombining binomial tree with up/down factors `u = exp(sigma*sqrt(dt))`, `d = 1/u`, and risk-neutral probability `p = (exp(r*dt) - d)/(u - d)`, then rolls the value back from expiry, taking the max of continuation value and immediate exercise at every node for the American case. This is the ground truth for the rest of the project: the NN's training labels and the RL environment's dynamics are both built from this same risk-neutral model.

As an independent check on the tree itself (rather than trusting it to grade its own convergence), `src/pricing/black_scholes.py` implements the closed-form European price. At 1,000 steps, the tree's European price matches Black-Scholes to within 0.01 for a standard ATM contract — this comparison is `tests/test_binomial.py::test_binomial_european_converges_to_black_schole`

5. The Neural-Network Method

A 2-hidden-layer MLP (`PutPricerNet`, 128 units/layer) was trained on 10,000 randomly sampled contracts (`S0 in [60,140]`, `K in [80,120]`, `T in [0.05, 2.0]`, `r in [0, 0.10]`, `sigma`

in [0.10, 0.50]), labeled with the 500-step binomial price. Inputs are standardized using training-set mean/std only. Training ran for 300 epochs with Adam, keeping the best-validation-MSE checkpoint. Best validation MSE: **0.0105**.

6. The RL Method

The RL environment (src/rl/env.py::AmericanPutEnv) uses a 3-feature state [time_fraction, time_to_expiry, moneyness] and the exact same risk-neutral dynamics as the binomial tree, for one fixed contract: S0=100, K=100, T=1.0, r=0.05, sigma=0.25, steps=50. The agent is a full DQN (src/rl/train_dqn.py) — target network, replay buffer, epsilon decaying 1.0 to 0.05 over 10,000 episodes.

7. Experimental Setup

- **Shared grid** (src/data/synthetic_contracts.py::make_shared_grid): S0 in {70,...,130} (step 10), K=100, T in {0.25, 0.5, 1.0, 2.0}, r in {0.02, 0.05}, sigma in {0.15, 0.25, 0.40}, steps=100 -> **168 contracts**.
- **NN vs. binomial**: every contract in the grid, priced both ways.
- **RL vs. binomial (single-contract)**: the contract the DQN was trained on, evaluated over 10,000 fresh episodes per policy, and over a 50-step x 81-moneyness grid for boundary agreement.
- **RL vs. binomial (full-grid, out-of-distribution check)**: the trained DQN and the always-hold baseline, run against all 168 grid contracts (300 fresh episodes each) — Section 8.4.
- **Seeds**: NN training used seed=42 throughout (sampling, split, weight init). DQN training used seed=42. Policy evaluation here uses fresh Monte Carlo seeds independent of training.

7.1 Hyperparameters

Neural-network pricer:

Hyperparameter	Value
Architecture	Linear(5,128) -> ReLU -> Linear(128,128) -> ReLU -> Linear(128,1)
Input features	[S0, K, T, r, sigma], standardized on training-set mean/std
Loss / optimizer	MSE / Adam, lr=1e-3
Batch size	256
Epochs	300 (best-validation checkpoint kept)
Train/val/test split	80/10/10, seed=42
Training set	10,000 contracts, labeled at 500 binomial steps
Best validation MSE	0.0105

RL exercise policy:

Hyperparameter	Value
Architecture	Linear(3,64) -> ReLU -> Linear(64,64) -> ReLU -> Linear(64,2)
State	[time_fraction, time_to_expiry, moneyness]

Hyperparameter	Value
Algorithm	DQN, target network + replay buffer
Optimizer	Adam, lr=1e-3, grad norm clipped to 5.0
Loss	Huber (smooth_l1)
Batch size	128
Replay buffer	50,000
Episodes	10,000
Target update	every 250 gradient steps
Epsilon schedule	1.0 -> 0.05, decay 0.999/episode
Training contract	S0=100, K=100, T=1.0, r=0.05, sigma=0.25, steps=50
Seed	42

Binomial pricer: steps set per use — 500 for NN labels, 1000 for the Black-Scholes convergence check, 100 for the shared grid, 50 to match the RL environment.

8. Results

8.1 Pricing accuracy — NN vs. binomial (full grid, n=168)

Metric	Value
MAE	0.0832
RMSE	0.1035
Max absolute error	0.2829
Mean bias	-0.0132
Median absolute error	0.0706
Mean relative error	1.334
Intrinsic-value violations	24 / 168
Spot-monotonicity violations	0

The mean relative error looks large in isolation, but it's driven by contracts whose true price is close to zero (deep OTM, short maturity) — a tiny absolute error becomes a large relative one against a near-zero denominator. MAE and RMSE are the more meaningful numbers given the price range in the grid (roughly $0-$60).

By moneyness:

Bucket	n	NN MAE	NN RMSE	NN bias	Binomial mean	NN mean
ITM put	72	0.0910	0.1116	-0.0063	21.97	21.96
ATM	48	0.1044	0.1229	-0.0317	6.66	6.63
OTM put	48	0.0503	0.0608	-0.0050	2.81	2.80

By maturity:

Bucket	n	NN MAE	NN RMSE	NN bias
Long (T>0.5y)	84	0.0733	0.0931	+0.0006
Short (T<=0.5y)	84	0.0931	0.1129	-0.0270

By volatility:

Bucket	n	NN MAE	NN RMSE	NN bias
High vol (sigma>0.20)	112	0.0853	0.1023	+0.0050
Low vol (sigma<=0.20)	56	0.0791	0.1058	-0.0495

The network is most accurate on deep OTM puts and slightly weaker near the money and on short-dated, low-volatility contracts — likely because these have smaller absolute price levels and sharper curvature near expiry.

24 of 168 contracts (14%) had NN predictions fall (slightly) below intrinsic value, expected since the network minimizes squared price error rather than being explicitly constrained to respect the intrinsic-value floor.

8.2 RL policy comparison (single contract, 10,000 evaluation episodes)

Policy	Value	Std. error	Exercise rate	Avg. exercise step
always-hold-to-expiry	7.4479	0.1098	0.000	-
dqn	**6.2106**	0.0435	0.787	18.6
random	0.9742	0.0222	1.000	1.0
immediate-exercise	0.0000	0.0000	1.000	0.0
binomial (American)	**7.9520**	-	-	-

8.3 Boundary agreement

The DQN's hold/exercise decision agreed with the binomial-optimal decision implied by the exercise boundary on **62.6%** of a 50-step x 81-moneyness grid of states.

8.4 RL policy evaluated across the full shared grid

Section 8.2 only evaluates the DQN on the contract it was trained on. To test whether it learned anything general about early exercise, the same trained policy (no retraining) was run against all 168 grid contracts, 300 fresh episodes each:

Metric (full grid, n=168)	Value
Mean DQN value	10.35
Mean always-hold value	12.08
Mean binomial (American) price	12.12
Mean gap (DQN - binomial)	-1.77
Mean absolute gap	1.77
Fraction of contracts where DQN beats always-hold	20.2%

By moneyness:

Bucket	Mean DQN value	Mean hold value	Mean binomial price	Mean abs. gap	n
ITM put	20.00	21.36	21.97	1.97	72

Bucket	Mean DQN value	Mean hold value	Mean binomial price	Mean abs. gap	n
ATM	4.61	7.12	6.66	2.06	48
OTM put	1.62	3.11	2.81	1.18	48

The DQN underperforms the binomial benchmark on essentially every slice of the grid, and beats the naive always-hold baseline on only about 1 in 5 contracts — proportionally worse than on its own training contract. This is expected: the network was trained on one fixed contract's dynamics and one fixed step count (50), and this check runs it against different strikes, maturities, volatilities, and step counts (100) it never saw. This is a genuine, not cherry-picked, measure of how little the policy generalizes.

9. Discussion: Strengths, Failures, and Limitations

NN pricer: strong overall accuracy (MAE ~0.08 against prices up to ~$60) with no monotonicity violations and only mild intrinsic-value violations. Weakest near the money and for short-dated contracts.

RL policy — the honest result: the DQN's value (6.21) is below both the binomial optimum (7.95) and the naive always-hold baseline (7.45), and its decisions only agree with the true optimal boundary 62.6% of the time. This means the DQN, despite a full replay-buffer + target-network setup, hasn't fully converged to the optimal exercise policy for even its one training contract. Plausible causes: 10,000 episodes with a single terminal reward per episode is a sparse-reward setting; more episodes, reward shaping, or a lower final epsilon might close the gap.

Generalization — tested, not just flagged: the DQN was trained against one contract's dynamics. Section 8.4 tests this directly: run against the full 168-contract grid, the same policy beats always-hold on only 20% of contracts and sits, on average, 1.77 away from the binomial optimum — a larger relative gap than on its own training contract. The policy has learned a rule for one contract, and that rule doesn't transfer. Extending it to generalize would require training a contract-conditioned agent (adding `S0, K, T, r, sigma` to the state, or training across a distribution of contracts instead of one) — future work, not something this project attempts.

Modeling simplifications common to all three methods: discrete time steps rather than continuous exercise opportunity; no dividends; no transaction costs; constant, known volatility (no smile or stochastic vol); a single fixed risk-free rate. The NN's labels come directly from the binomial tree, so it can only be as correct as its teacher — it approximates the binomial model faster, it doesn't outperform it. The RL policy's evaluation is Monte Carlo, so the reported values carry real standard error rather than being exact.

10. Reproducibility

```
pip install -r requirements.txt
pip install -e .
python -m evaluation.comparison
python -m pytest tests/ -v
```

Both trained checkpoints are already committed under `reports/results/`, so retraining is optional. All seeds are fixed (`seed=42`); the Monte Carlo policy-evaluation numbers will vary slightly run to run, within the reported standard error.

11. Conclusion

Bringing the binomial tree, the NN pricer, and the RL policy together onto one shared, honestly-reported evaluation gives a genuinely mixed picture: the NN pricer is a strong, fast approximation to the binomial benchmark across a wide range of contracts, while the RL policy — though structurally capable of learning a state-dependent exercise rule, unlike the fixed baselines — hasn't converged to the true optimal policy for even the single contract it was trained on, and demonstrably doesn't generalize when tested directly against the full grid. That measured generalization gap is the most important limitation for anyone building on this next.

Repository: `AayushGupta34/SOC-26-American-Put-Option-Pricing` **Figures:** `reports/figures/` — payoff diagram, exercise boundary, NN-vs-binomial scatter, learned exercise region (`exercise_region.png`), NN learning curve.